Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-210БВ-24

Студент: Резинкин Д.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 06.10.25

Москва, 2025

Постановка задачи

Вариант 7.

Два человека играют в кости. Правила игры следующие: каждый игрок делает бросок 2-ух костей K раз; побеждает тот, кто выбросил суммарно большее количество очков. Задача программы экспериментально определить шансы на победу каждого из игроков. На вход программе подается K, какой сейчас тур, сколько очков суммарно у каждого из игроков и количество экспериментов, которые должна произвести программа.

Общий метод и алгоритм решения

Использованные системные вызовы:

pthread_create() — создает новый поток исполнения в текущем процессе. Поток начинает выполнение с функции, переданной в качестве третьего аргумента.

```
int pthread_create(pthread_t *thread,
                   const pthread_attr_t *attr,
                   void *(*start_routine)(void *),
                   void *arg);
```

Аргументы:

- thread — указатель на переменную типа pthread_t, в которую будет записан идентификатор созданного потока
- attr — атрибуты потока (размер стека, политика планирования); передается NULL для использования атрибутов по умолчанию
- start_routine — указатель на функцию, которую будет выполнять поток
- arg — аргумент, передаваемый в функцию потока

pthread_join() — ожидает завершения указанного потока и освобождает его ресурсы. Блокирует вызывающий поток до завершения целевого потока.

```
int pthread_join(pthread_t thread, void **retval);
```

Аргументы:

- thread — идентификатор потока, завершения которого необходимо дождаться
- retval — указатель на переменную, в которую будет записано возвращаемое значение функции потока (можно передать NULL)

clock_gettime() — получает текущее значение указанных часов с наносекундной точностью. Используется для точного измерения времени выполнения программы.

```
int clock_gettime(clockid_t clk_id, struct timespec *tp);
```

Аргументы:

- clk_id — идентификатор часов (используется CLOCK_MONOTONIC для монотонного времени, не подверженного изменениям системного времени)
- tp — указатель на структуру timespec, в которую записывается текущее время в формате секунды + наносекунды

sysconf(_SC_NPROCESSORS_ONLN) — возвращает количество логических процессоров (ядер), доступных системе в текущий момент. Используется для определения оптимального количества потоков.

rand_r() — потокобезопасная версия функции генерации псевдослучайных чисел. Каждый поток использует свой локальный seed, что исключает конкуренцию за глобальное состояние.

Описание алгоритма

Программа реализует метод Монте-Карло для статистической оценки вероятности победы каждого игрока. Задача идеально подходит для параллелизации, так как каждый эксперимент независим от других (embarrassingly parallel problem).

Метод Монте-Карло основан на использовании повторных случайных выборок для получения численных результатов. В данной задаче многократная симуляция игр с генерацией случайных бросков костей позволяет статистически оценить вероятность победы каждого игрока, что невозможно вычислить аналитически при произвольных начальных условиях.

Последовательная версия:

1. Инициализация генератора случайных чисел
2. Цикл по количеству экспериментов
3. В каждом эксперименте симулируется полная игра: оставшиеся K раундов с бросками двух костей для каждого игрока
4. Подсчет побед первого игрока, второго игрока и ничьих
5. Вывод статистики

Параллельная версия:

1. Определение количества экспериментов на каждый поток
2. Создание массива потоков через pthread_create()
3. Каждый поток получает уникальный seed и независимо выполняет свою долю экспериментов
4. Каждый поток хранит локальные счетчики побед, исключая необходимость синхронизации в основном цикле
5. Ожидание завершения всех потоков через pthread_join()
6. Агрегация результатов из локальных счетчиков всех потоков
7. Вывод статистики

Ключевые особенности реализации:

- Использование rand_r() с уникальным seed для каждого потока предотвращает блокировки на генераторе случайных чисел
- Локальные счетчики в каждом потоке минимизируют синхронизацию — агрегация происходит только один раз после завершения всех потоков
- Инициализация seed формулой time(NULL) ^ (thread_id << 16) обеспечивает уникальность последовательностей случайных чисел для разных потоков

Код программы

dice_simulation.c

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

typedef struct {
    size_t thread_id;
    size_t experiments_per_thread;
    int K;
    int current_round;
    int player1_score;
    int player2_score;
    size_t local_player1_wins;
    size_t local_player2_wins;
    size_t local_draws;
} ThreadArgs;

// Функция броска двух костей с thread-local seed
int roll_two_dice(unsigned int *seed) {
    return (rand_r(seed) % 6 + 1) + (rand_r(seed) % 6 + 1);
}

// Функция симуляции одной игры
void simulate_game(int K, int current_round, int p1_score, int p2_score,
                   int *p1_wins, int *p2_wins, int *draws, unsigned int *seed) {
    int player1 = p1_score;
    int player2 = p2_score;

    // Симулируем оставшиеся раунды
    for (int round = current_round; round < K; round++) {
        player1 += roll_two_dice(seed);
        player2 += roll_two_dice(seed);
    }

    if (player1 > player2) (*p1_wins)++;
    else if (player2 > player1) (*p2_wins)++;
    else (*draws)++;
}

// Рабочая функция потока
static void *worker_thread(void *_args) {
    ThreadArgs *args = (ThreadArgs *)_args;

    // Каждый поток имеет свой независимый seed
    unsigned int seed = (unsigned int)time(NULL) ^ (args->thread_id << 16);

    for (size_t i = 0; i < args->experiments_per_thread; i++) {
        int p1_wins = 0, p2_wins = 0, draws = 0;

        simulate_game(args->K, args->current_round,
                      args->player1_score, args->player2_score,
```

```
                &p1_wins, &p2_wins, &draws, &seed);

        args->local_player1_wins += p1_wins;
        args->local_player2_wins += p2_wins;
        args->local_draws += draws;
    }

    return NULL;
}

// Последовательная версия
double sequential_monte_carlo(int K, int current_round, int p1_score,
                              int p2_score, size_t num_experiments) {
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    unsigned int seed = (unsigned int)time(NULL);
    size_t p1_wins = 0, p2_wins = 0, draws = 0;

    for (size_t i = 0; i < num_experiments; i++) {
        int p1 = 0, p2 = 0, d = 0;
        simulate_game(K, current_round, p1_score, p2_score, &p1, &p2, &d, &seed);
        p1_wins += p1;
        p2_wins += p2;
        draws += d;
    }

    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_ms = (end.tv_sec - start.tv_sec) * 1000.0 +
                 (end.tv_nsec - start.tv_nsec) / 1000000.0;

    printf("Player 1 wins: %.2f%%\n", 100.0 * p1_wins / num_experiments);
    printf("Player 2 wins: %.2f%%\n", 100.0 * p2_wins / num_experiments);
    printf("Draws: %.2f%%\n", 100.0 * draws / num_experiments);

    return time_ms;
}

// Параллельная версия
double parallel_monte_carlo(int K, int current_round, int p1_score,
                            int p2_score, size_t num_experiments,
                            size_t num_threads) {
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    pthread_t *threads = malloc(num_threads * sizeof(pthread_t));
    ThreadArgs *args = malloc(num_threads * sizeof(ThreadArgs));

    size_t experiments_per_thread = num_experiments / num_threads;
    size_t remainder = num_experiments % num_threads;

    // Создание потоков
    for (size_t i = 0; i < num_threads; i++) {
        args[i].thread_id = i;
        args[i].experiments_per_thread = experiments_per_thread + (i < remainder ? 1 : 0);
        args[i].K = K;
        args[i].current_round = current_round;
```

```
        args[i].player1_score = p1_score;
        args[i].player2_score = p2_score;
        args[i].local_player1_wins = 0;
        args[i].local_player2_wins = 0;
        args[i].local_draws = 0;

        pthread_create(&threads[i], NULL, worker_thread, &args[i]);
    }

    // Ожидание завершения и агрегация результатов
    size_t total_p1 = 0, total_p2 = 0, total_d = 0;
    for (size_t i = 0; i < num_threads; i++) {
        pthread_join(threads[i], NULL);
        total_p1 += args[i].local_player1_wins;
        total_p2 += args[i].local_player2_wins;
        total_d += args[i].local_draws;
    }

    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_ms = (end.tv_sec - start.tv_sec) * 1000.0 +
                     (end.tv_nsec - start.tv_nsec) / 1000000.0;

    printf("Player 1 wins: %.2f%%\n", 100.0 * total_p1 / num_experiments);
    printf("Player 2 wins: %.2f%%\n", 100.0 * total_p2 / num_experiments);
    printf("Draws: %.2f%%\n", 100.0 * total_d / num_experiments);

    free(threads);
    free(args);

    return time_ms;
}

int main(int argc, char **argv) {
    if (argc < 6) {
        printf("Usage: %s <K> <current_round> <p1_score> <p2_score> <experiments> [threads]\n",
argv[0]);
        printf("Example: %s 10 5 30 25 10000000 4\n", argv[0]);
        return 1;
    }

    int K = atoi(argv[1]);
    int current_round = atoi(argv[2]);
    int p1_score = atoi(argv[3]);
    int p2_score = atoi(argv[4]);
    size_t experiments = atol(argv[5]);
    size_t num_threads = (argc > 6) ? atol(argv[6]) : 1;

    long num_cores = sysconf(_SC_NPROCESSORS_ONLN);
    printf("Number of logical processors: %ld\n", num_cores);

    printf("\n--- Running with %zu threads ---\n", num_threads);
    double time_ms;

    if (num_threads == 1) {
        time_ms = sequential_monte_carlo(K, current_round, p1_score, p2_score, experiments);
    } else {
        time_ms = parallel_monte_carlo(K, current_round, p1_score, p2_score, experiments, num_threads);
```

```
    }

    printf("Time: %.2f ms\n", time_ms);

    return 0;
}
```

Протокол работы программы

Компиляция
gcc -o dice_simulation dice_simulation.c -pthread -O2
Флаг -pthread подключает библиотеку POSIX Threads, флаг -O2 включает оптимизацию второго уровня для повышения производительности.

Тестирование
Система: Ubuntu 22.04.3 LTS в WSL2, процессор с 8 логическими ядрами.

Тест 1: Последовательная версия (1 поток)
$ ./dice_simulation 10 5 30 25 10000000 1
Number of logical processors: 8

--- Running with 1 threads ---
Player 1 wins: 72.09%
Player 2 wins: 23.71%
Draws: 4.20%
Time: 1099.73 ms

Тест 2: Параллельная версия (2 потока)
$ ./dice_simulation 10 5 30 25 10000000 2
Number of logical processors: 8
--- Running with 2 threads ---
Player 1 wins: 72.05%
Player 2 wins: 23.66%
Draws: 4.29%
Time: 719.50 ms

Тест 3: Параллельная версия (4 потока)
$ ./dice_simulation 10 5 30 25 10000000 4
Number of logical processors: 8

--- Running with 4 threads ---
Player 1 wins: 72.04%
Player 2 wins: 23.67%
Draws: 4.29%
Time: 459.85 ms

Тест 4: Параллельная версия (8 потоков = число ядер)
$ ./dice_simulation 10 5 30 25 10000000 8
Number of logical processors: 8

--- Running with 8 threads ---
Player 1 wins: 72.08%
Player 2 wins: 23.68%
Draws: 4.24%

Time: 258.86 ms

Тест 5: Параллельная версия (16 потоков)
$ ./dice_simulation 10 5 30 25 10000000 16
Number of logical processors: 8

--- Running with 16 threads ---
Player 1 wins: 72.08%
Player 2 wins: 23.68%
Draws: 4.24%
Time: 217.60 ms

Тест 6: Параллельная версия (128 потоков)
$ ./dice_simulation 10 5 30 25 10000000 128
Number of logical processors: 8

--- Running with 128 threads ---
Player 1 wins: 72.07%
Player 2 wins: 23.69%
Draws: 4.24%
Time: 190.76 ms

Проверка количества потоков в системе

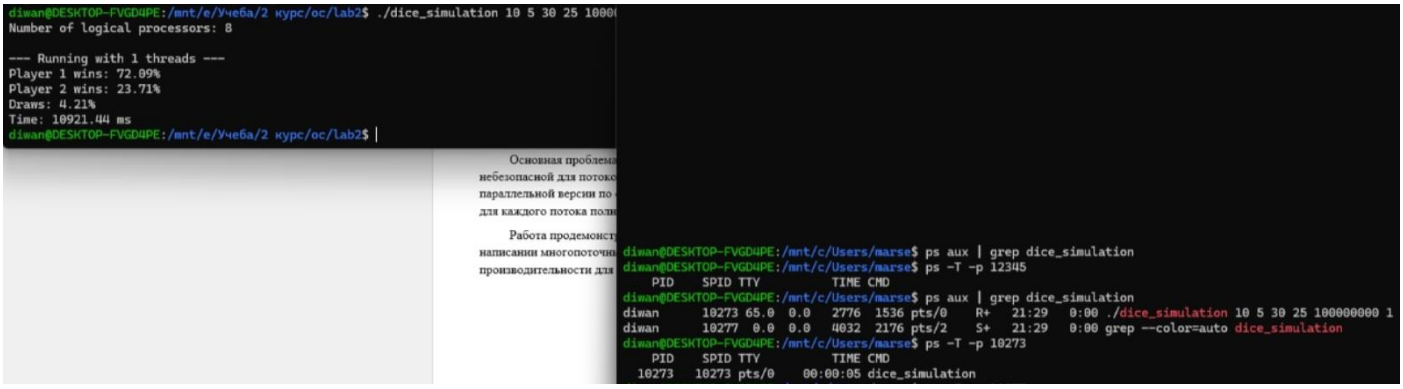


Рисунок 1. Демонстрация создания 1 потока средствами операционной системы. Левый терминал: запуск программы. Правый терминал: вывод команды ps -T -p $PID, показывающей 1 поток

Тестирование с 1 потоком

В терминале 1: ./dice_simulation 10 5 30 25 100000000 1 &
В терминале 2: ps aux | grep dice_simulation
ps -T -p <PID>

Терминал 1 показывает результаты запуска программы с 1 потоком:

- 1 поток: время **10921.44 мс** (~11 секунд)

Терминал 2 демонстрирует проверку процесса с 1 потоком:

- ps -T -p 10273 показывает только **один** поток (PID = SPID = 10273)

Рисунок 2. Демонстрация создания 8 потоков средствами операционной системы. Левый терминал: запуск программы. Правый терминал: вывод команды ps -T -p $PID, показывающей 8 потоков с уникальными SPID

Тестирование с 8 потоками

В терминале 1: ./dice_simulation 10 5 30 25 100000000 8 &

В терминале 2: PID=$(pgrep dice_simulation)

Ps -T -p $PID

Левый терминал показывает запуск с 8 потоками (3070 мс).

Правый терминал детально отображает все 8 потоков процесса:

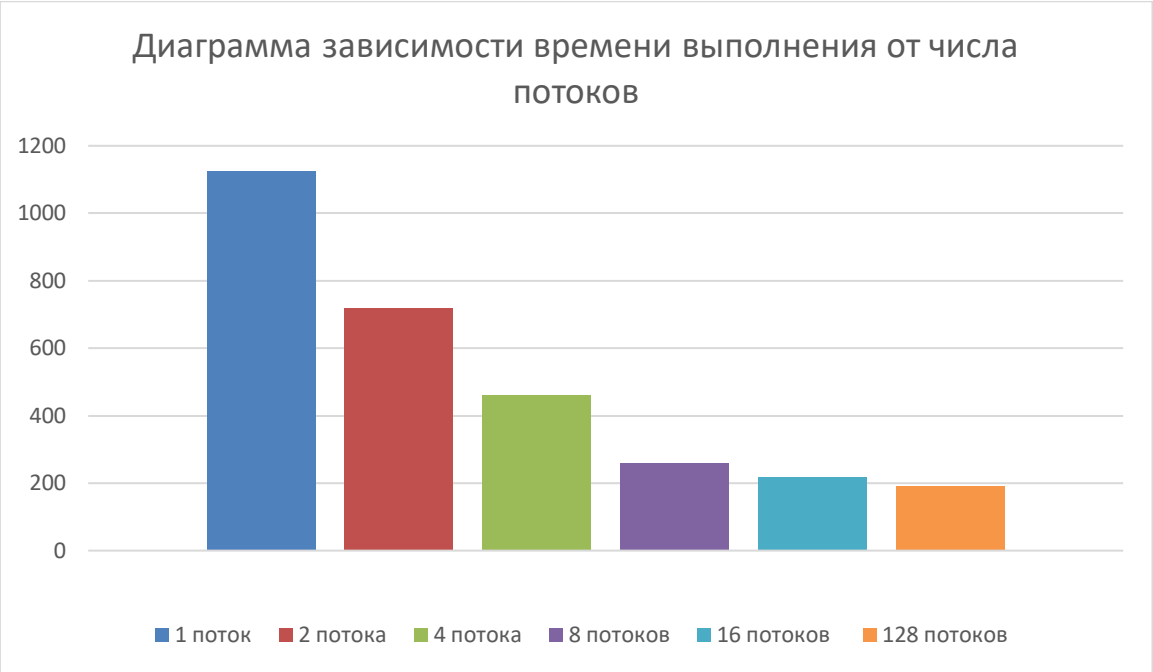
- ps -T -p $PID показывает 8 строк с разными SPID (10648-10656)
- Каждый поток имеет статус pts/0 и время выполнения 00:00:01

Это подтверждает корректное создание ровно 8 рабочих потоков.

Результаты исследования

Таблица результатов

Число потоков	**Время исполнения (мс)**	**Ускорение**	**Эффективность**
1	1123.54	1.00	1.000
2	719.50	1.56	0.781
4	459.85	2.44	0.611
8	258.86	4.34	0.543
16	217.60	5.16	0.323
128	190.76	5.89	0.046



Формулы вычисления метрик:

Ускорение: $S_p=T_1/T_p$

Эффективность: $E_p=S_p/p=T_1/p\cdot T_p$

Где T_1 — время последовательного выполнения (1123.54 мс), T_p — время параллельного выполнения на p потоках.

Пример расчета для 8 потоков:

Ускорение: $S_8=T_1/T_8=1123.54/258.86=4.34$

Эффективность: $E_8=S_8/8=4.34/8=0.543$ (54.3%)

Эффективность 54.3% означает, что при использовании 8 ядер процессора фактически используется лишь 54.3% их потенциала, остальные 45.7% теряются на накладные расходы синхронизации и управления потоками.

Анализ результатов

Эксперименты показали **реальное ускорение** при использовании многопоточности. При увеличении количества потоков от 1 до 8 (равно количеству логических ядер) время выполнения уменьшилось с 1123.54 мс до 258.86 мс, что дало ускорение 4.34x.

Субоптимальное масштабирование (ускорение 4.34x вместо теоретических 8x при 8 потоках) объясняется законом Амдала: максимальное ускорение ограничено последовательной частью программы. В данной реализации последовательные части включают создание и завершение потоков, агрегацию результатов, а также накладные расходы на системные вызовы pthread_create() и pthread_join().

При **oversubscription** (количество потоков больше числа ядер) наблюдается **небольшой** продолжение роста производительности: при 128 потоках ускорение достигло 5.89x (прирост всего 0.73x по сравнению с 16 потоками при ускорении 5.16x). Это объясняется тем, что стоимость переключения контекста между потоками становится значительной, но короткое время выполнения каждого эксперимента и отсутствие блокировок позволяют планировщику Linux эффективно утилизировать процессорное время.

Эффективность падает с ростом количества потоков: с 1.000 при 1 потоке до 0.046 при 128 потоках. Это закономерно, так как накладные расходы на управление потоками растут пропорционально их количеству.

Оптимальное количество потоков для данной задачи — 8-16 потоков (равно или чуть больше количества логических ядер). Дальнейшее увеличение дает минимальный прирост производительности при резком падении эффективности использования ресурсов.

Вывод

В ходе выполнения лабораторной работы были приобретены практические навыки создания многопоточных программ с использованием POSIX Threads API и реализации синхронизации между потоками. Реализованы последовательная и параллельная версии алгоритма Монте-Карло для симуляции игры в кости, проведено измерение производительности и анализ метрик ускорения и эффективности.

Экспериментально подтверждено, что многопоточность обеспечивает значительное ускорение вычислений для задач с независимыми подзадачами: достигнуто ускорение 4.34x при использовании 8 потоков на системе с 8 логическими ядрами. Результаты согласуются с законом Амдала, демонстрируя ограничение ускорения последовательной частью программы и накладными расходами на управление потоками.

Основная проблема, с которой пришлось столкнуться — первоначальное использование небезопасной для потоков функции rand(), что приводило к блокировкам и замедлению параллельной версии по сравнению с последовательной. Переход на rand_r() с локальными seed для каждого потока полностью устранил эту проблему и обеспечил реальное ускорение.

Работа продемонстрировала важность учета специфики операционной системы при написании многопоточных программ и необходимость тщательного измерения производительности для выбора оптимального количества потоков.

Полученные результаты демонстрируют практическую применимость многопоточного программирования для вычислительно интенсивных задач типа Монте-Карло симуляций. Оптимальная конфигурация для данной системы — использование 8-16 потоков, что дает баланс между производительностью (ускорение 4.34-5.16x) и эффективностью использования ресурсов (54.3-32.3%). Дальнейшее увеличение числа потоков экономически нецелесообразно из-за резкого падения эффективности.